

Workshop No. 1

Requirements, User Stories and Story Mapping

Spynet



Watching every part | Auditing every weakness

Grupo 5 — Los Extraditables

Juan Diego Rozo Alvarez	Backend
Santiago Alejandro Rojas Feo	Backend
Alejandro Medina Rojas	Frontend
Misael Jesus Florez Anave	DevOps
Cristian David Arcia Quintero	Testing / Documentacion
Marlon David Pabon Munoz	Testing / Documentacion

1. Requirements Documentation

Spynet es una aplicación web diseñada para analizar y mapear la composición tecnológica de páginas web. Dada una URL de entrada, el sistema identifica las tecnologías utilizadas (frameworks frontend, backend, servidores), obtiene información del dominio (DNS, WHOIS, geolocalización de IP) y permite consultar el historial del sitio mediante la Wayback Machine. Los resultados se almacenan en una base de datos relacional y se exponen a través de una API REST, habilitando comparaciones y reutilización de datos.

1.1 Requisitos Funcionales

Los requisitos funcionales describen las capacidades que el sistema debe ofrecer a sus actores: el **Usuario** (quien ingresa URLs y consulta resultados) y el **Sistema** (que procesa solicitudes y gestiona datos).

ID	Nombre	Descripción
RF-01	Análisis de sitio web	El sistema debe aceptar una URL como entrada y ejecutar un análisis automático que detecte las tecnologías usadas por el sitio (frontend, backend, servidores, CDN).
RF-02	Detección de tecnologías frontend	El sistema debe identificar frameworks de frontend como React, Vue, Angular, Next.js y Nuxt.js, mediante análisis del HTML, etiquetas script y headers HTTP.
RF-03	Detección de tecnologías backend	El sistema debe identificar tecnologías de backend como Django, Node.js, Laravel, WordPress y PHP, a través de headers HTTP, cookies y patrones en el HTML.
RF-04	Detección de CDN y hosting	El sistema debe identificar el proveedor de CDN u hosting del sitio (Cloudflare, AWS, Vercel, Netlify) usando nameservers DNS, headers y rangos de IP.
RF-05	Consulta de registros DNS	El sistema debe consultar los registros DNS del dominio (A, AAAA, MX, NS, TXT) usando dnspython y almacenar la IP resuelta, nameservers y registros de correo.
RF-06	Consulta WHOIS del dominio	El sistema debe obtener datos WHOIS del dominio analizado: registrador, fecha de creación, fecha de expiración y calcular la antigüedad del dominio en años.
RF-07	Geolocalización de IP	El sistema debe geolocalizar la IP del dominio consultando una API pública (ip-api.com) y retornar país, ciudad, ISP y organización que administra esa dirección IP.
RF-08	Historial con Wayback Machine	El sistema debe consultar la API CDX de archive.org para obtener snapshots históricos del sitio: cantidad total de archivados y los últimos timestamps con URL de acceso.
RF-09	Almacenamiento de resultados	El sistema debe persistir cada análisis en base de datos relacional, incluyendo tecnologías detectadas, datos DNS, WHOIS, geolocalización y snapshots de Wayback.

RF-10	Consulta de historial de analisis	El sistema debe exponer un endpoint GET que liste todos los analisis realizados ordenados cronologicamente, con su URL, estado y cantidad de tecnologias detectadas.
RF-11	Detalle de analisis	El sistema debe permitir consultar el detalle completo de un analisis especifico por su ID, incluyendo todas las tecnologias con su evidencia de deteccion.
RF-12	Comparacion de sitios	El sistema debe comparar dos analisis e identificar tecnologias en comun, exclusivas del sitio A y exclusivas del sitio B.
RF-13	Estadisticas globales	El sistema debe exponer estadisticas: total de analisis, total de tecnologias detectadas y ranking de las 20 tecnologias mas frecuentes con su categoria.
RF-14	Manejo de errores	El sistema debe manejar fallos graciosamente: si un servicio externo (WHOIS, DNS, Wayback, GeoIP) falla, retorna null en ese campo sin interrumpir el resto del analisis.

1.2 Requisitos No Funcionales

Los requisitos no funcionales definen los atributos de calidad del sistema, relacionados con rendimiento, seguridad, mantenibilidad y portabilidad.

ID	Nombre	Descripcion
RNF-01	Rendimiento	El analisis completo de un sitio (tecnologias + DNS + WHOIS + GeoIP + Wayback) debe completarse en menos de 20 segundos bajo condiciones normales de red.
RNF-02	Disponibilidad	La API debe tener disponibilidad del 99% en entorno de produccion. Los fallos en servicios externos no deben derribar el sistema.
RNF-03	Escalabilidad	La arquitectura modular basada en patrones de diseno (Factory, Singleton, Facade) debe permitir agregar nuevos detectores o servicios sin modificar la logica existente.
RNF-04	Seguridad	El sistema no debe ejecutar el codigo de los sitios analizados. Las URLs son consultadas de forma pasiva (HTTP GET). No se almacenan credenciales ni datos sensibles.
RNF-05	Mantenibilidad	El codigo debe seguir principios OOP, tener cobertura de tests automatizados superior al 70% y documentacion clara de arquitectura y patrones usados.
RNF-06	Usabilidad de la API	La API REST debe seguir convenciones estandar: verbos HTTP correctos, codigos de estado apropiados (200, 201, 400, 404, 502) y respuestas JSON consistentes.
RNF-07	Portabilidad	El sistema debe poder ejecutarse en cualquier entorno con Python 3.10+ mediante Docker, garantizando reproducibilidad entre desarrollo y produccion.

RNF-08	Trazabilidad	Cada tecnologia detectada debe incluir el campo evidence que explica que patron o header origino la deteccion, facilitando la verificacion de resultados.
---------------	---------------------	---

2. User Stories

Las historias se redactan para los dos actores definidos en la propuesta del proyecto: **Usuario** (desarrollador, estudiante o investigador que analiza sitios web) y **Sistema** (actor interno responsable de procesar las solicitudes). El formato usado es: *Como [rol], quiero [accion] para que [beneficio]*, con criterios de aceptacion en formato Given / When / Then.

US-01	Analizar un sitio web	Usuario
Como usuario , quiero ingresar una URL y obtener el perfil tecnologico completo del sitio, para conocer que frameworks, servidores y servicios utiliza.		
Given	el usuario envia POST /api/analyze/ con {"url": "https://ejemplo.com"}	
When	el sistema procesa la solicitud correctamente	
Then	retorna JSON con tecnologias detectadas (nombre, categoria, confianza, evidencia), datos DNS, WHOIS, geolocalizacion y snapshots de Wayback, con status 201.	
Given	el usuario envia una URL sin protocolo como "github.com"	
When	el sistema recibe la solicitud	
Then	agrega automaticamente "https://" y procesa la URL sin retornar error.	
Given	el sitio no responde o se produce un timeout	
When	el sistema agota el tiempo de espera	
Then	retorna status 502 con mensaje descriptivo del error.	

US-02	Consultar registros DNS del dominio	Usuario
Como usuario , quiero ver los registros DNS del sitio analizado, para entender su infraestructura: IP, nameservers y proveedores de correo.		
Given	el sistema resuelve el dominio de la URL ingresada	
When	el DNSService consulta los registros usando dnspython	
Then	la respuesta incluye la IP (registro A), los nameservers (NS) y los registros de correo (MX) del dominio.	
Given	los nameservers contienen el patron "cloudflare.com"	
When	el sistema procesa los registros NS	
Then	el CDNDetector usa esa informacion para confirmar el uso de Cloudflare con un nivel de confianza de al menos 85%.	

US-03	Consultar datos WHOIS del dominio	Usuario
Como usuario , quiero ver los datos WHOIS del sitio analizado, para conocer quien lo registro, cuando fue creado y cuando expira.		

Given	el sistema completa el analisis de una URL valida
When	el WhoisService consulta los datos del dominio
Then	la respuesta incluye registrar, created_date, expiration_date y domain_age_years calculado automaticamente.
Given	el dominio tiene privacidad activada (WHOIS privacy)
When	el sistema intenta consultar los datos
Then	retorna los campos disponibles y marca como null los ocultos, sin interrumpir el resto del analisis.

US-04	Ver geolocalizacion de la IP del sitio	Usuario
Como usuario , quiero saber en que pais y con que proveedor esta alojado el sitio analizado, para entender donde esta fisicamente su infraestructura.		
Given	el sistema resuelve la IP del dominio	
When	consulta la API publica de ip-api.com con esa IP	
Then	la respuesta incluye pais, ciudad, ISP y organizacion asociados a esa IP.	
Given	la API de geolocalizacion no esta disponible	
When	se produce un error de conexion o timeout	
Then	el sistema retorna null en el campo geo y continua con el resto del analisis sin interrupciones.	

US-05	Ver historial de snapshots con Wayback Machine	Usuario
Como usuario , quiero ver cuantas veces ha sido archivado el sitio en Wayback Machine y en que fechas, para entender su historial y evolucion en el tiempo.		
Given	el usuario analiza cualquier URL	
When	el WaybackService consulta la CDX API de archive.org	
Then	la respuesta incluye snapshot_count (total de archivados) y los ultimos 5 timestamps con su URL de acceso a la version archivada.	
Given	el sitio no tiene historial en Wayback Machine	
When	la API devuelve una lista vacia	
Then	el sistema retorna snapshot_count: 0 y snapshots: [] sin generar error.	

US-06	Consultar historial de analisis	Usuario
Como usuario , quiero ver todos los analisis realizados previamente, para no repetir analisis de sitios ya consultados y reutilizar la informacion.		
Given	existen analisis almacenados en la base de datos	
When	el usuario hace GET /api/analyses/	

Then	recibe lista ordenada por fecha descendente con id, url, status, duration_ms y cantidad de tecnologías detectadas.
Given	no se han realizado analisis aun
When	el usuario consulta GET /api/analyses/
Then	recibe lista vacia [] con codigo 200.

US-07	Ver detalle completo de un analisis	Usuario
Como usuario , quiero consultar el detalle completo de un analisis especifico, incluyendo la evidencia de deteccion de cada tecnologia.		
Given	existe un analisis con id=1	
When	el usuario hace GET /api/analyses/1/	
Then	recibe el analisis completo con todas las tecnologias y su campo evidence explicando que patron o header origino cada deteccion.	
Given	el id solicitado no existe en la base de datos	
When	el usuario consulta GET /api/analyses/999/	
Then	el sistema retorna 404 con mensaje {"error": "Analisis no encontrado"}.	

US-08	Comparar las tecnologias de dos sitios	Usuario
Como usuario , quiero comparar las tecnologias de dos sitios analizados, para identificar similitudes y diferencias en sus stacks tecnologicos.		
Given	existen dos analisis con id=1 e id=2	
When	el usuario hace GET /api/compare/?a=1&b=2	
Then	recibe JSON con tres listas: common (tecnologias en comun), only_a (exclusivas del sitio A) y only_b (exclusivas del sitio B).	
Given	alguno de los ids enviados no existe	
When	el usuario consulta el endpoint con un id invalido	
Then	el sistema retorna 404 indicando cual de los dos analisis no fue encontrado.	

US-09	Consultar estadisticas globales	Usuario
Como usuario , quiero ver cuales son las tecnologias mas detectadas en todos los analisis, para identificar tendencias del ecosistema web.		
Given	existen multiples analisis almacenados	
When	el usuario hace GET /api/stats/	
Then	recibe total_analyses, total_techs y top_technologies: ranking de las 20 mas frecuentes con nombre, categoria y conteo.	

US-10	Orquestar todos los servicios desde un punto unico (Facade)	Sistema
Como sistema , debo orquestar FrontendDetector, BackendDetector, CDNDetector, DNSService, WhoisService, GeoService y WaybackService desde el metodo analyze(), compartiendo la peticion HTTP entre los detectores que la necesiten.		
Given	el Analyzer recibe una URL para analizar	
When	ejecuta el metodo analyze() aplicando el Facade Pattern	
Then	realiza exactamente una peticion HTTP al sitio, la comparte entre FrontendDetector, BackendDetector y CDNDetector, y llama independientemente a DNSService, WhoisService, GeoService y WaybackService, consolidando todo en un JSON.	
Given	uno de los servicios externos falla durante el analisis	
When	se produce una excepcion en WhoisService, GeoService o WaybackService	
Then	el sistema registra el error en logs, retorna null en ese campo especifico y completa el resto del analisis con normalidad.	

US-11	Detectar tecnologias frontend sin falsos positivos	Sistema
Como sistema , debo detectar frameworks de frontend analizando HTML, scripts y headers, aplicando un sistema de puntaje para evitar falsos positivos.		
Given	el HTML de un sitio contiene id="root" y un script con react.production.min.js	
When	el FrontendDetector evalua las firmas de React	
Then	detecta React con confidence >= 60 y registra la evidencia especifica que origino la deteccion.	
Given	el HTML contiene solo un patron debil de Angular como main.js	
When	el FrontendDetector evalua las firmas de Angular	
Then	no reporta Angular porque requiere al menos 2 senales fuertes para evitar falsos positivos.	

3. User Story Mapping

El mapa organiza las historias de usuario en tres niveles: **Actividades** (objetivos de alto nivel del usuario segun la propuesta), **Tareas** (acciones concretas que las componen) e **Historias por sprint** (priorizacion iterativa del desarrollo). Se planifican tres sprints alineados con las funcionalidades de la propuesta.

Analizar sitios web	Dominio e infraestructura	Historial y monitoreo	Comparar y estadísticas
Ingresar URL Detectar frontend/backend Detectar CDN/hosting	Consultar DNS (A, MX, NS) Consultar WHOIS Geolocalizacion de IP	Snapshots Wayback Machine Historial de archivado Evolucion del sitio	Comparar dos analisis Estadísticas globales Ver historial de analisis
Sprint 1 Base del sistema	US-10: Facade (orquestrador) US-01: Analizar URL	US-02: Registros DNS US-03: Datos WHOIS	—
Sprint 2 Funcionalidads avanzadas	US-07: Detalle de analisis US-11: Deteccion frontend	US-04: Geolocalizacion IP	US-05: Wayback Machine
Sprint 3 CI/CD y documentacion	RF-14: Manejo de errores Tests automatizados	Panel admin Django	Docker + GitHub Actions Documentacion final

Las celdas con "—" indican que esa actividad no tiene historia asignada en ese sprint. Sprint 3 incluye CI/CD con GitHub Actions y Docker segun lo especificado en la propuesta.

3.1 Priorizacion MoSCoW

Las historias se clasifican segun el metodo MoSCoW: **Must have** (imprescindible para la entrega), **Should have** (importante pero no critico), **Could have** (deseable si hay tiempo) y **Won't have** (fuera del alcance actual).

ID	Historia	Prioridad	Sprint
US-01	Analizar un sitio web (tecnologias)	Must have	Sprint 1
US-02	Consultar registros DNS del dominio	Must have	Sprint 1
US-03	Consultar datos WHOIS del dominio	Must have	Sprint 1
US-04	Ver geolocalizacion de la IP	Should have	Sprint 2
US-05	Ver historial Wayback Machine	Should have	Sprint 2
US-06	Consultar historial de analisis	Must have	Sprint 1

US-07	Ver detalle completo de un analisis	Must have	Sprint 2
US-08	Comparar tecnologias de dos sitios	Should have	Sprint 2
US-09	Consultar estadisticas globales	Should have	Sprint 2
US-10	Facade: orquestar todos los servicios	Must have	Sprint 1
US-11	Deteccion frontend sin falsos positivos	Must have	Sprint 1

4. Referencias

- [1] Django Project. (2024). *Django documentation v4.2*. <https://docs.djangoproject.com/en/4.2/>
- [2] Django REST Framework. (2024). *DRF documentation*. <https://www.django-rest-framework.org/>
- [3] Internet Archive. (2024). *Wayback Machine CDX API*. https://archive.org/help/wayback_api.php
- [4] ip-api.com. (2024). *IP Geolocation API documentation*. <https://ip-api.com/docs>
- [5] Patton, J. (2014). *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media.
- [6] Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.
- [7] plan.io. (2024). *User Story Mapping: A Complete Guide*. <https://plan.io/blog/user-story-mapping/>
- [8] Wappalyzer. (2024). *Open source technology profiler*. <https://github.com/wappalyzer/wappalyzer>
- [9] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.